Recursion & Memoization

Dev Karan Singh (devkaran1231)
Expert at codeforces (1817)
5 star at codechef (2040)

Today's plan

- Print increasing number from 1 to n
- Print decreasing number from n to 1
- Print factorial of a number
- Check if given string is a palindrome or not
- Linear search
- Nth fibonacci number (memoization)
- Print subsequence
- Count subsequence with given sum

Next class (23rd)

function

```
function (){
[                ]
}
```

Body of the function

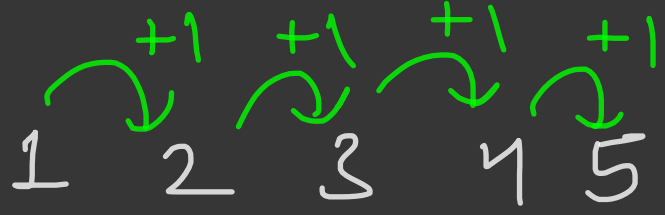
recursive function → a function calling itself in it's body.

Base case

```
— function(){
    [          ]   Base case
    [function()]   Body of the function
}
```

Q print increasing numbers from 1 to n

n=5



i=1

Steps → check base
print i
call (i+1)

Base case →

```
if (i == n){
    cout i;
    return;
}
```

```
void f(int i, int n){
    if(i==n){
        cout<<i<<endl;
        return;
    }
    cout<<i<<" ";
    f(i+1, n)
}
```

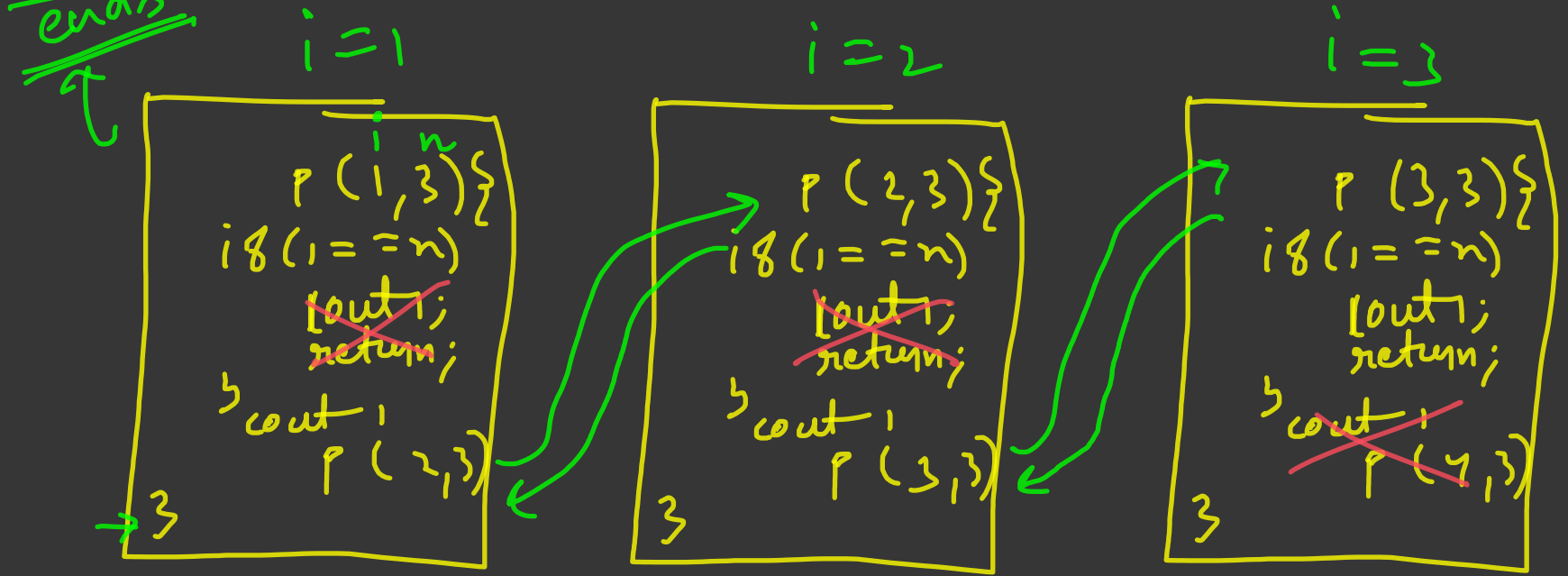
Base case

recursive step

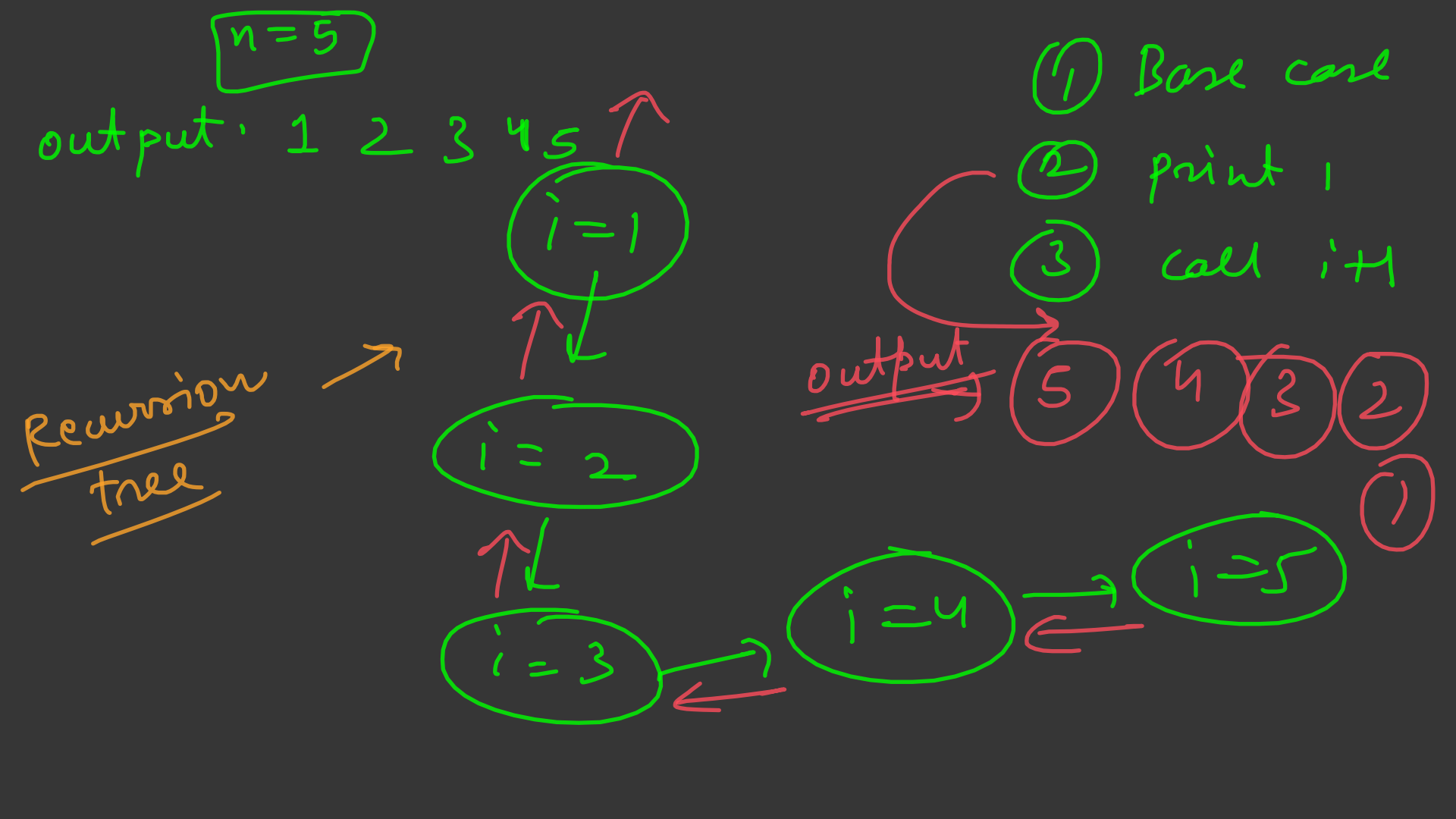
stack space

$n = 3$

recursion ends



output : 1 2 3


n = 5
output : 1 2 3 4 5
Recursion tree
i = 1
i = 2
i = 3
i = 4
i = 5
1 Base case
2 print i
3 call i+1
output
5
4
3
2
1

Q Print number decreasing from n to 1.

n1

n = 5

steps → ① Base case

② print i

③ call i-1

output : 5 4 3 2 1

i ↘ i-1

Base case → if (n == 0) {

return;

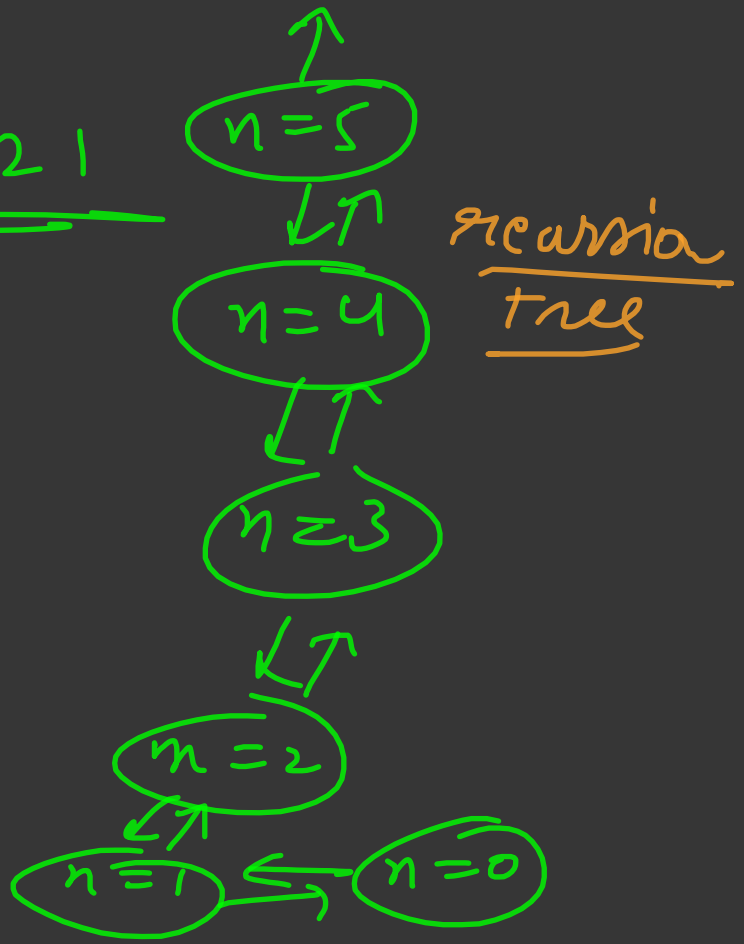
}

code:

```
void f(int n){
    if(n==0){
        return n;
    }
    cout << n << " ";
    f(n-1);
}
```

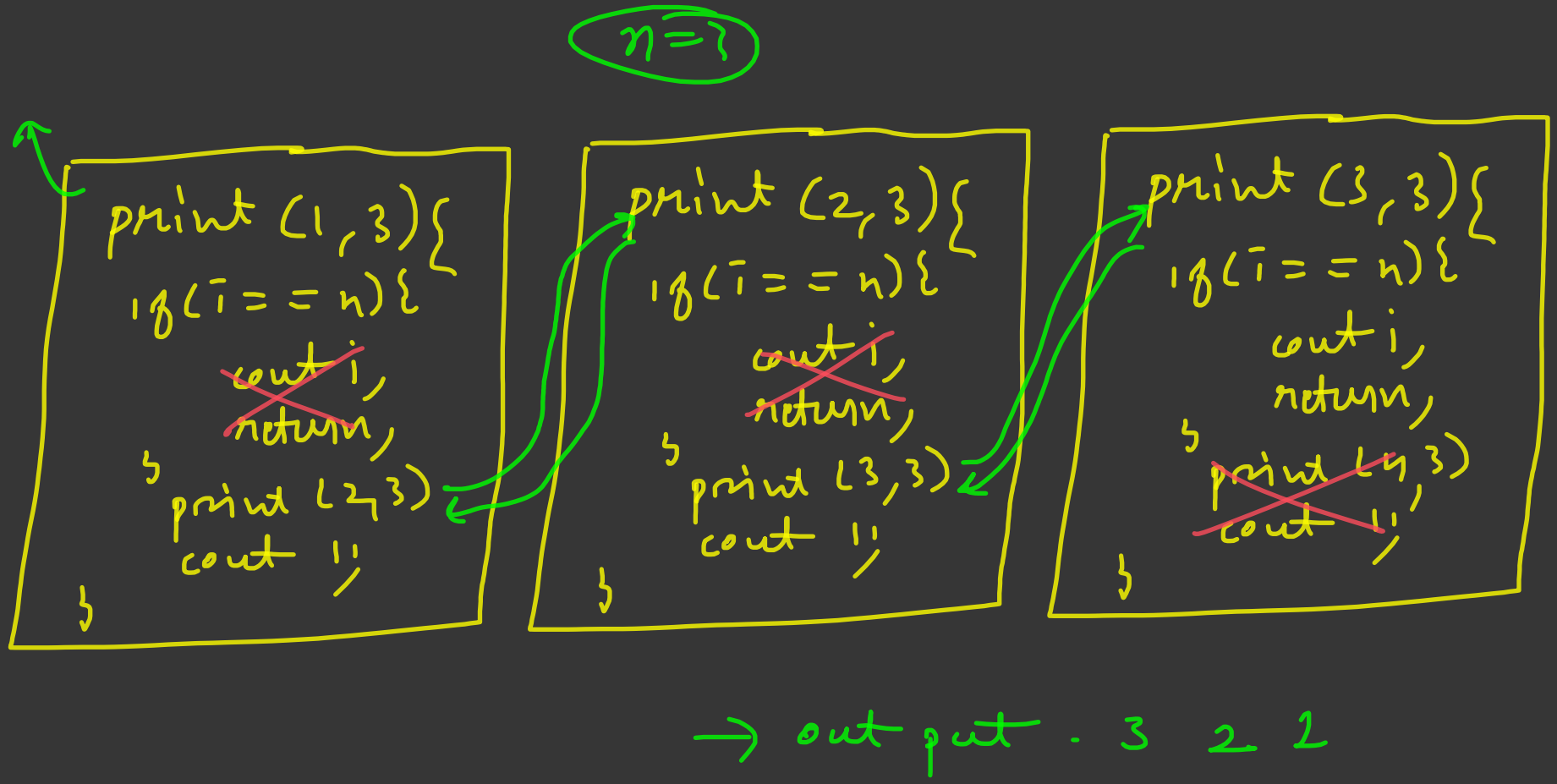
output:

5 4 3 2 1



M2

```
void f(int i, int n){
    if(i == n){
        cout i;
        return;
    }
    f(i+1);
    cout i;
}
```


n = 3
print (1, 3){
if (i == n){
cout i;
return;
}
print (2,3)
cout ;;
}
print (2, 3){
if (i == n){
cout i;
return;
}
print (3,3)
cout ;;
}
print (3, 3){
if (i == n){
cout i;
return;
}
print (4,3)
cout ;;
}

→ output : 3 2 1

Q. return factorial of a number

Hint: return type will be int

0! = 1

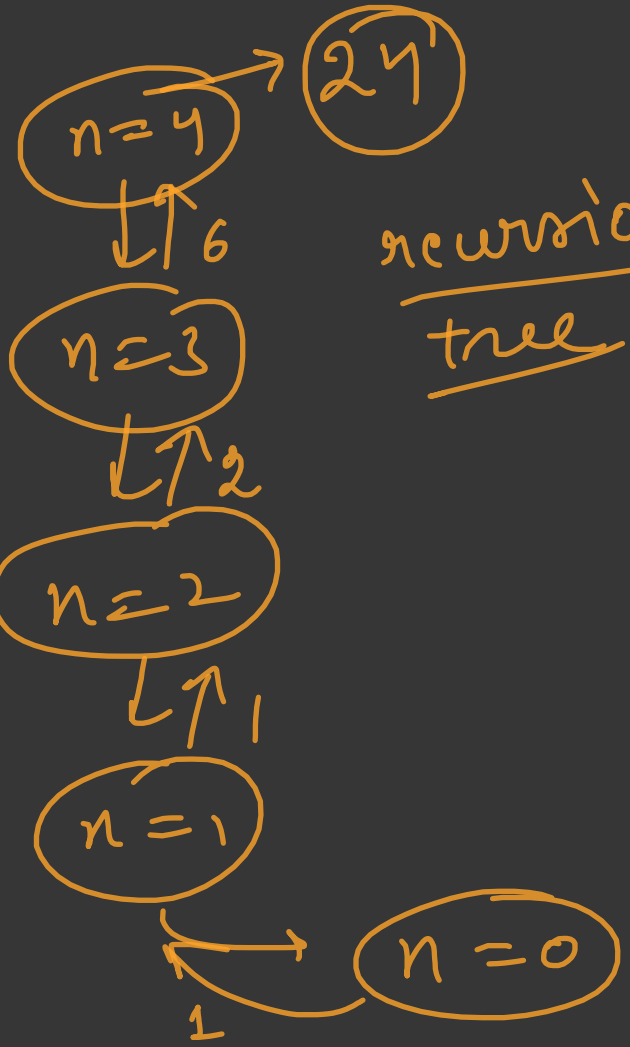
1! = 0! x 1

2! = 1! x 2

3! = 2! x 3 → 6

base case →

```
if(n==0){
  return 1;
}
```

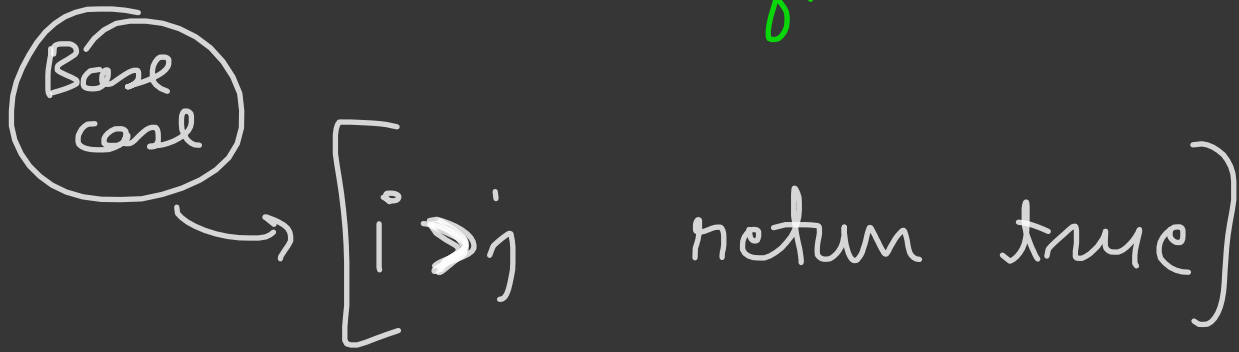


recursion tree

```
int fact(int n){
    if (n == 0){
        return 1;
    }
    return fact(n-1)*n;
}
```

Q check palindrome recursively.

string that is same after reversing it.



i=0

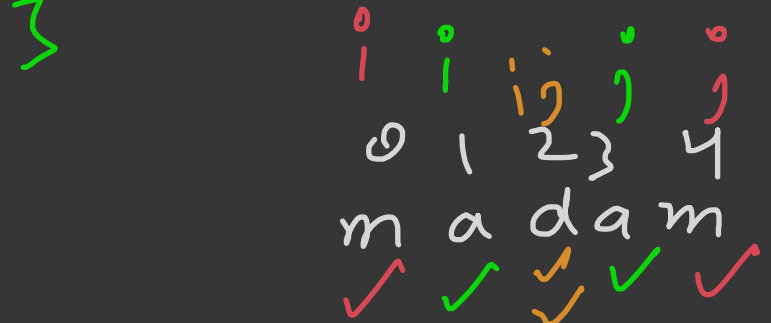
j=n-1

i

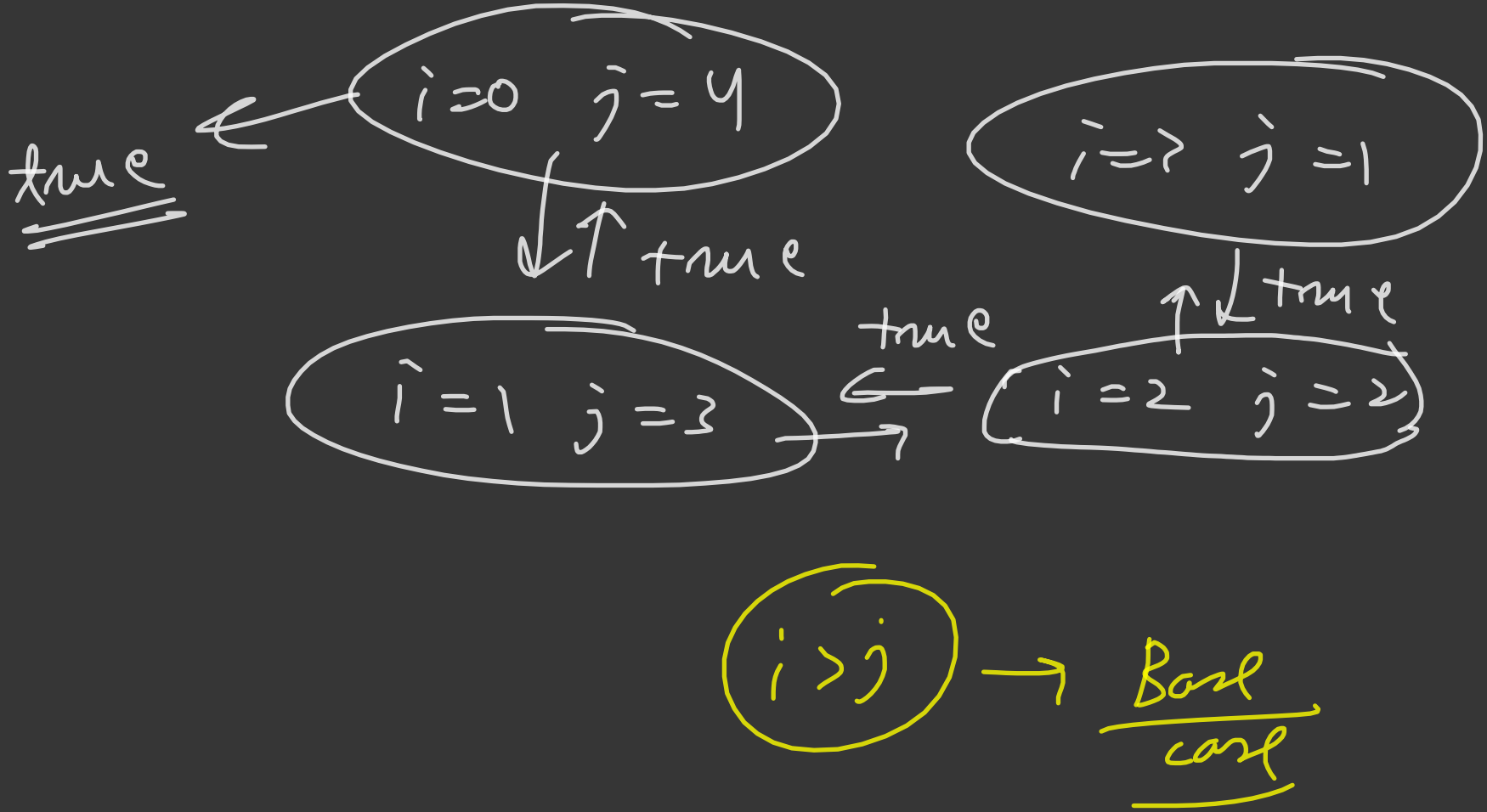
m a d a m

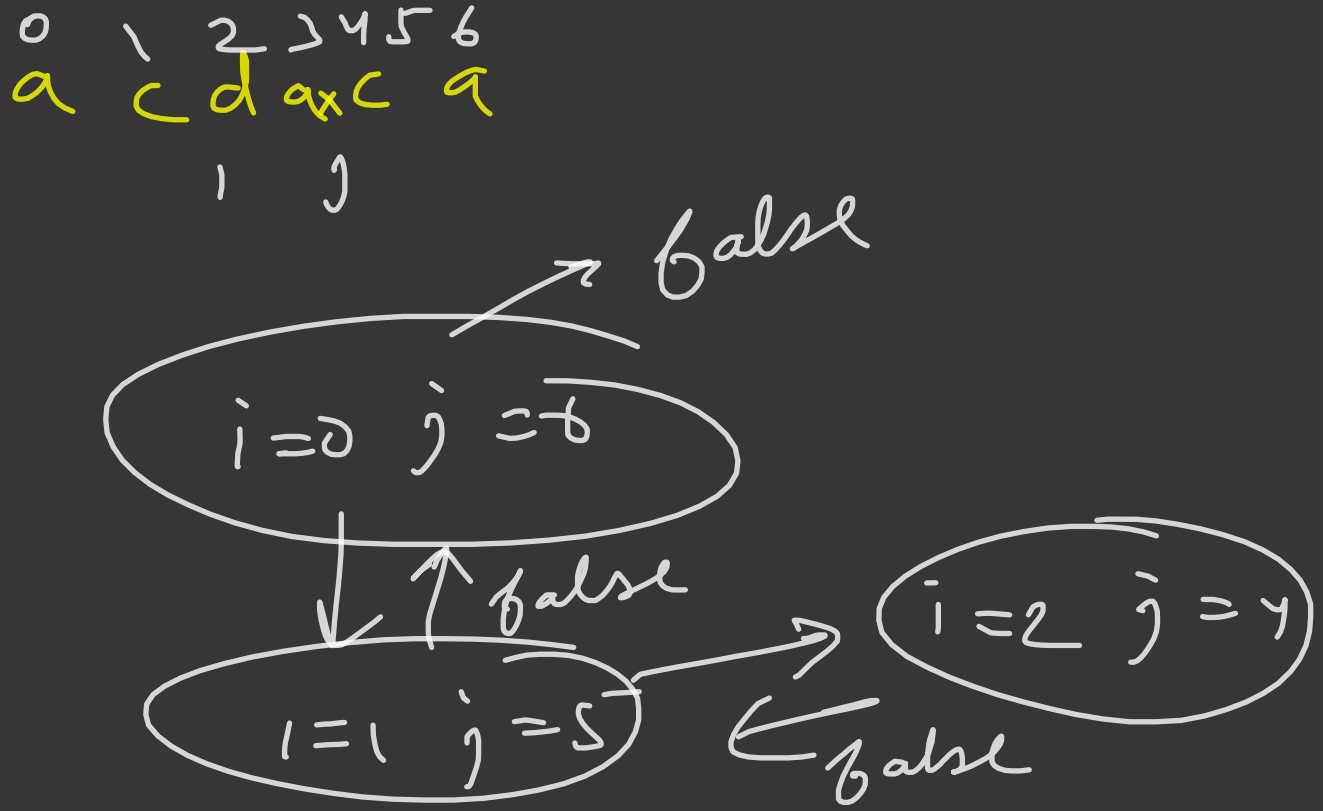
j

```
bool func(int i, int j){
    if (j<i) return true;
    if (s[i] != s[j]) return false;
    return func(i+1, j-1);
}
```



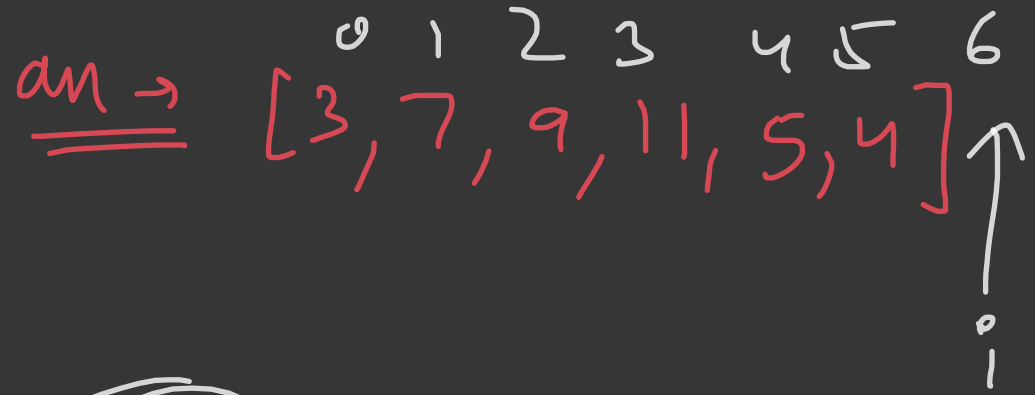
j < i

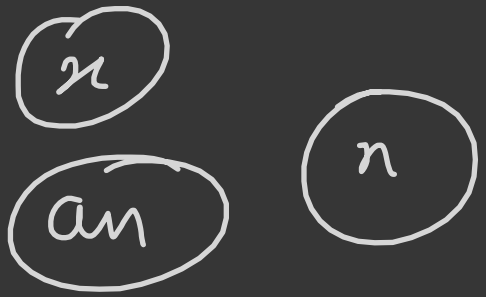

i=0 j=4
true
true
i=1 j=3
true
i=2 j=2
true
i=2 j=1
true
i>j
Base case

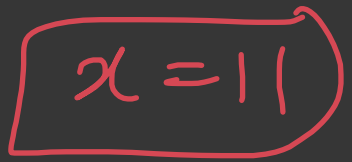

0 1 2 3 4 5 6
a c d a x c a
i j
false
i=0 j=6
false
i=1 j=5
i=2 j=4
false

Que

Linear search









x → target

code

int

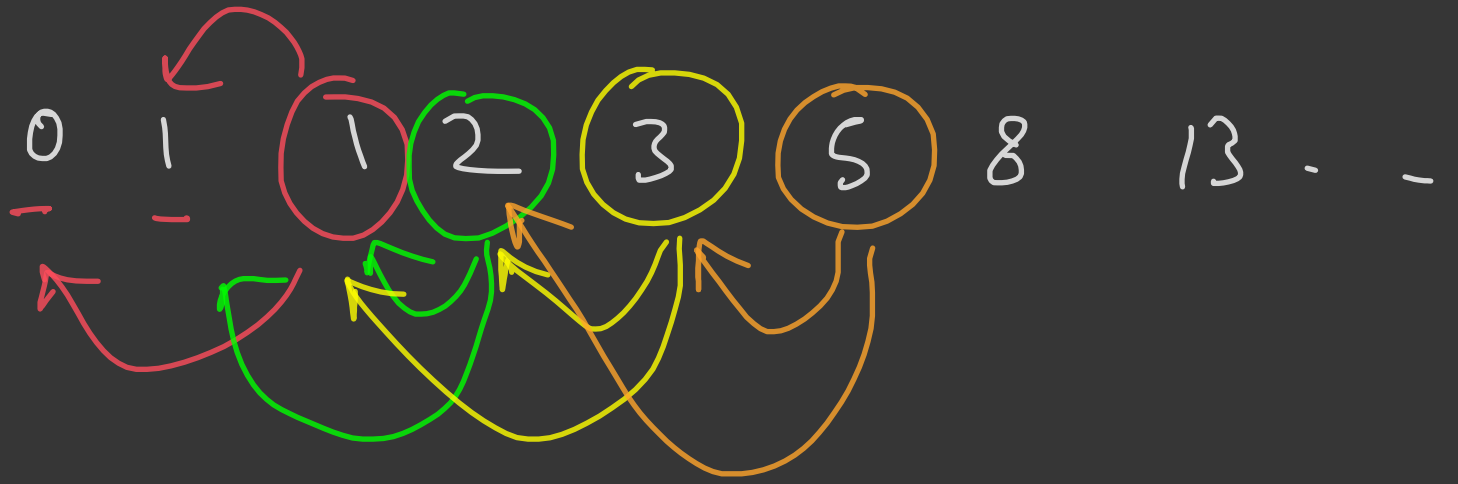
```
bool find( int i, int n, int an[],
                                int x){
    if(i==n){
        return false;
    }
    if(an[i]==x) return true;
    return find(i+1, n, an, x);
}
```

-1

0

1

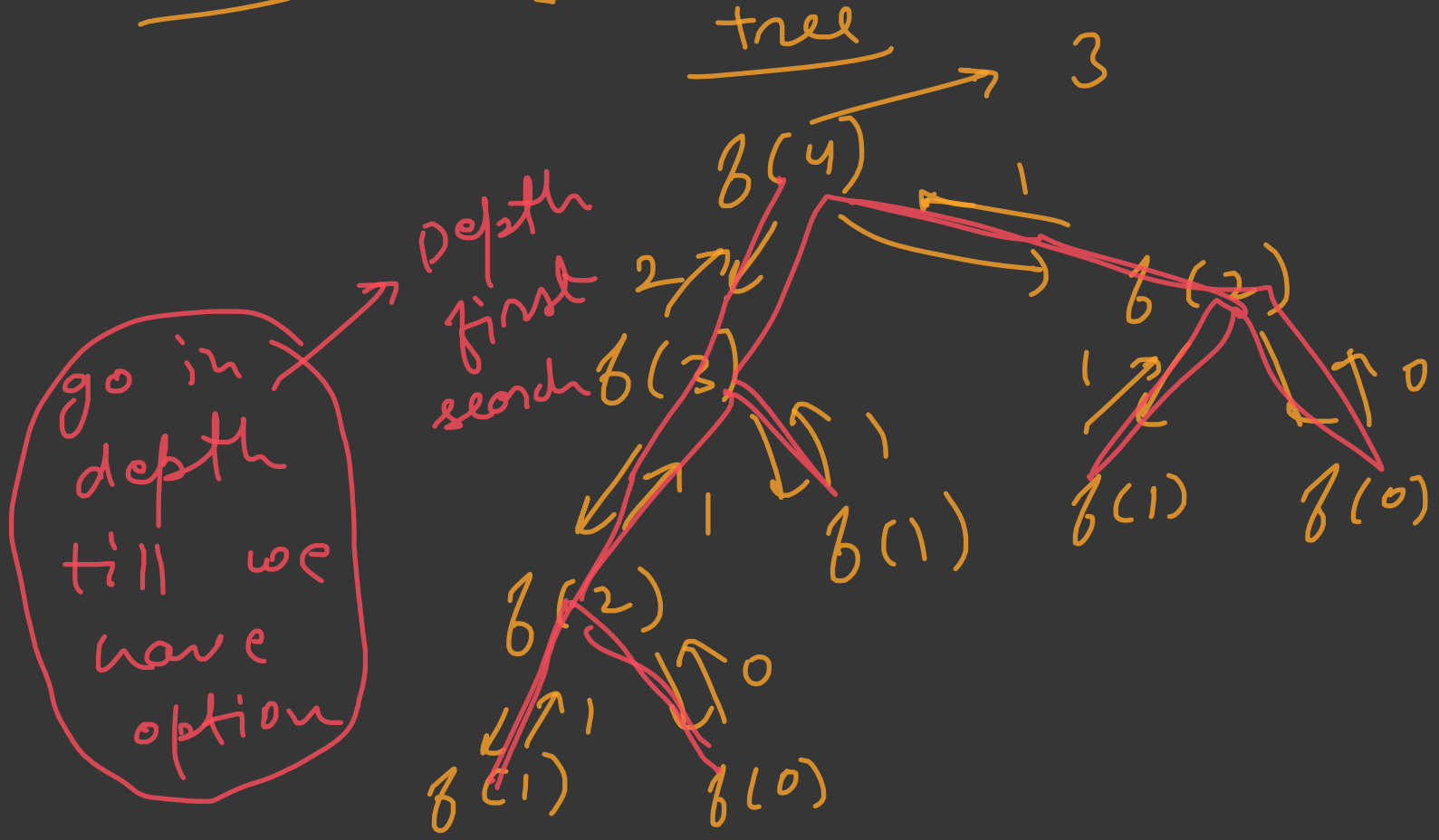
Q return n^{th} fibonnaci number

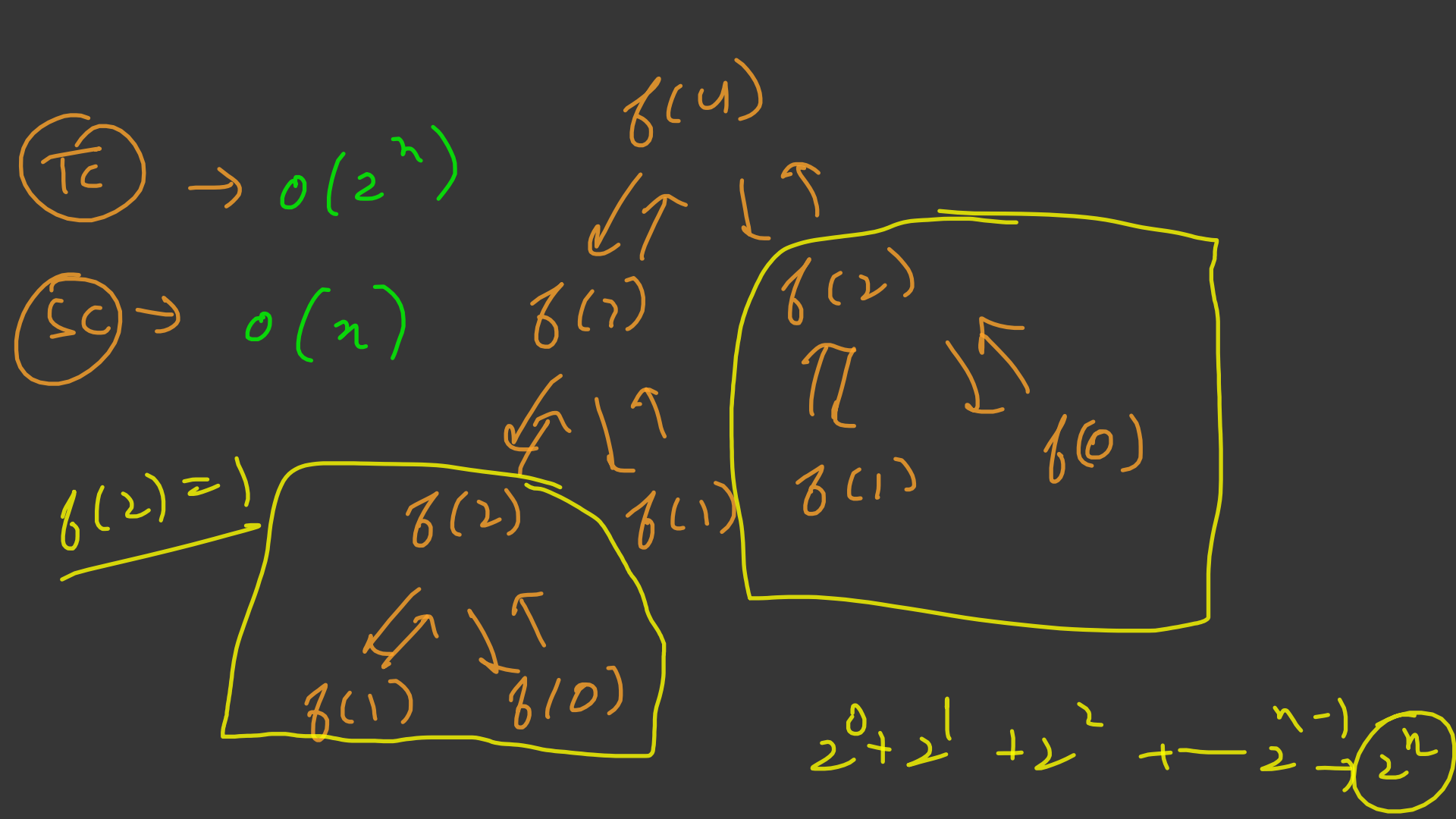


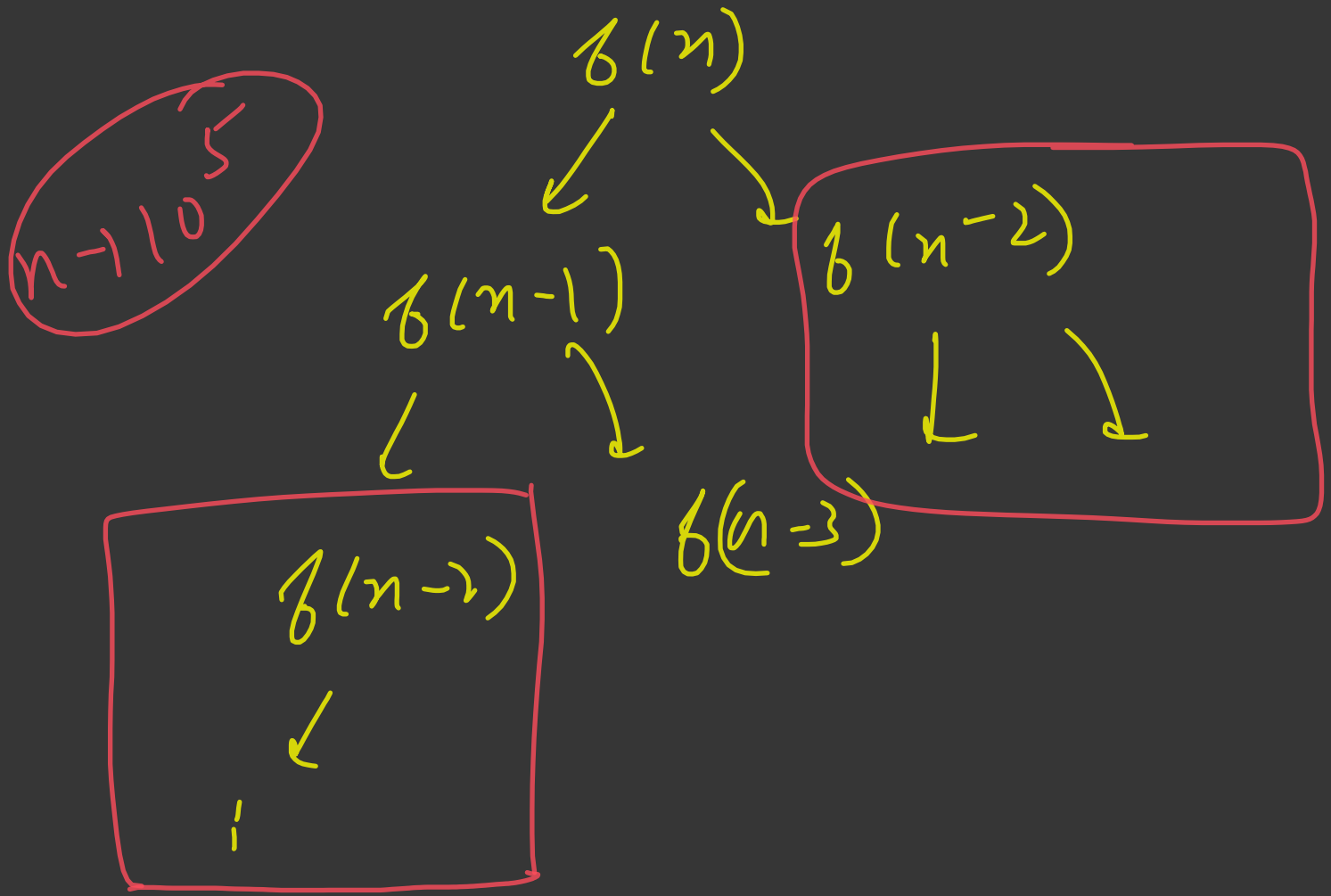

```
int fibo(int n){
    if(n<=1) return n;
    return fibo(n-1) + fibo(n-2);
}
```

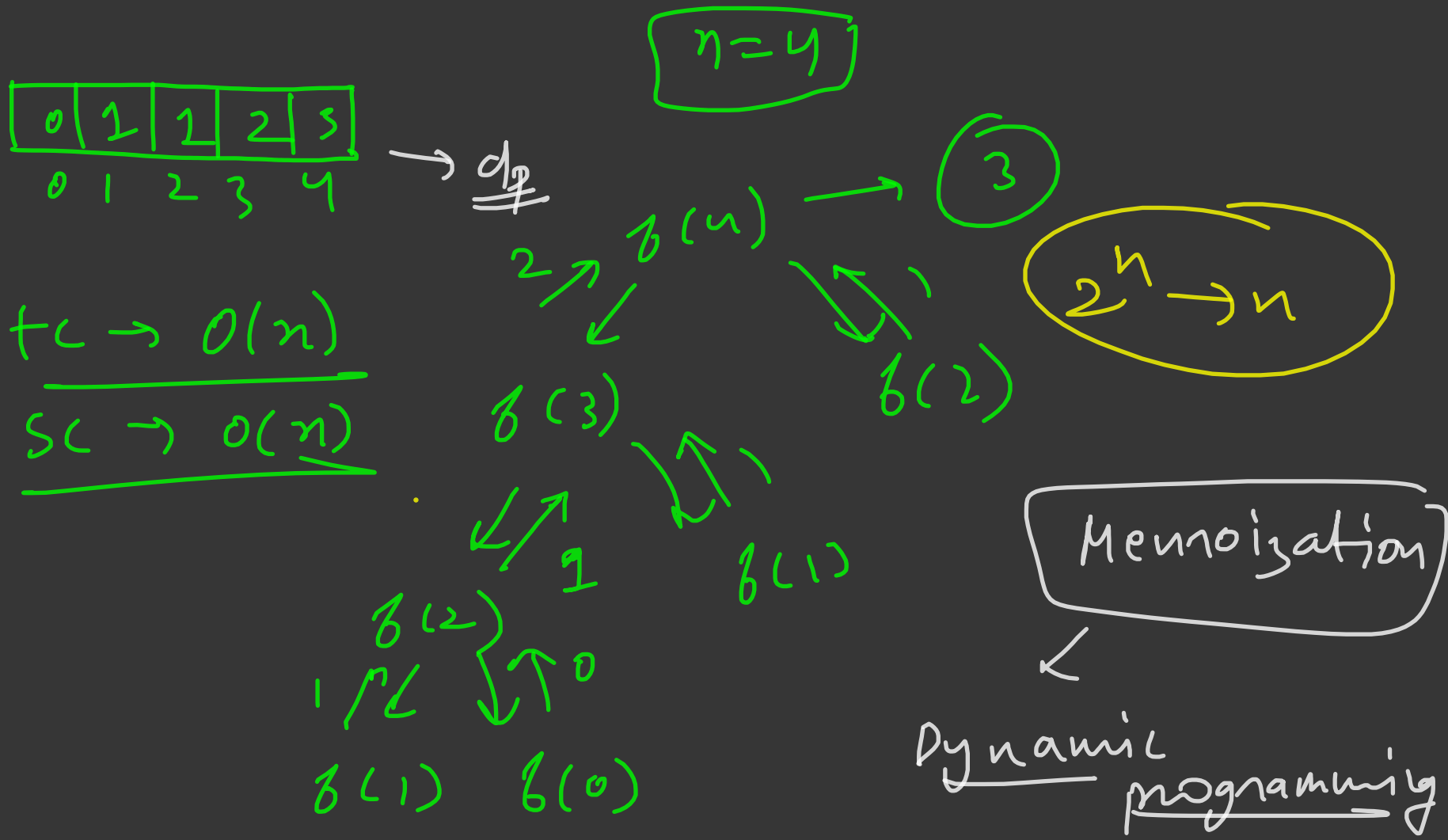
n=4

recursion tree




TC → O(2^n)
SC → O(n)
f(4)
f(3)
f(2)
f(2)=1
f(2)
f(1)
f(1)
f(0)
f(1)
f(0)
2^0 + 2^1 + 2^2 + ... + 2^(n-1) → 2^n


f(n)
n-1 10^5
f(n-1)
f(n-2)
f(n-2)
f(n-3)
1

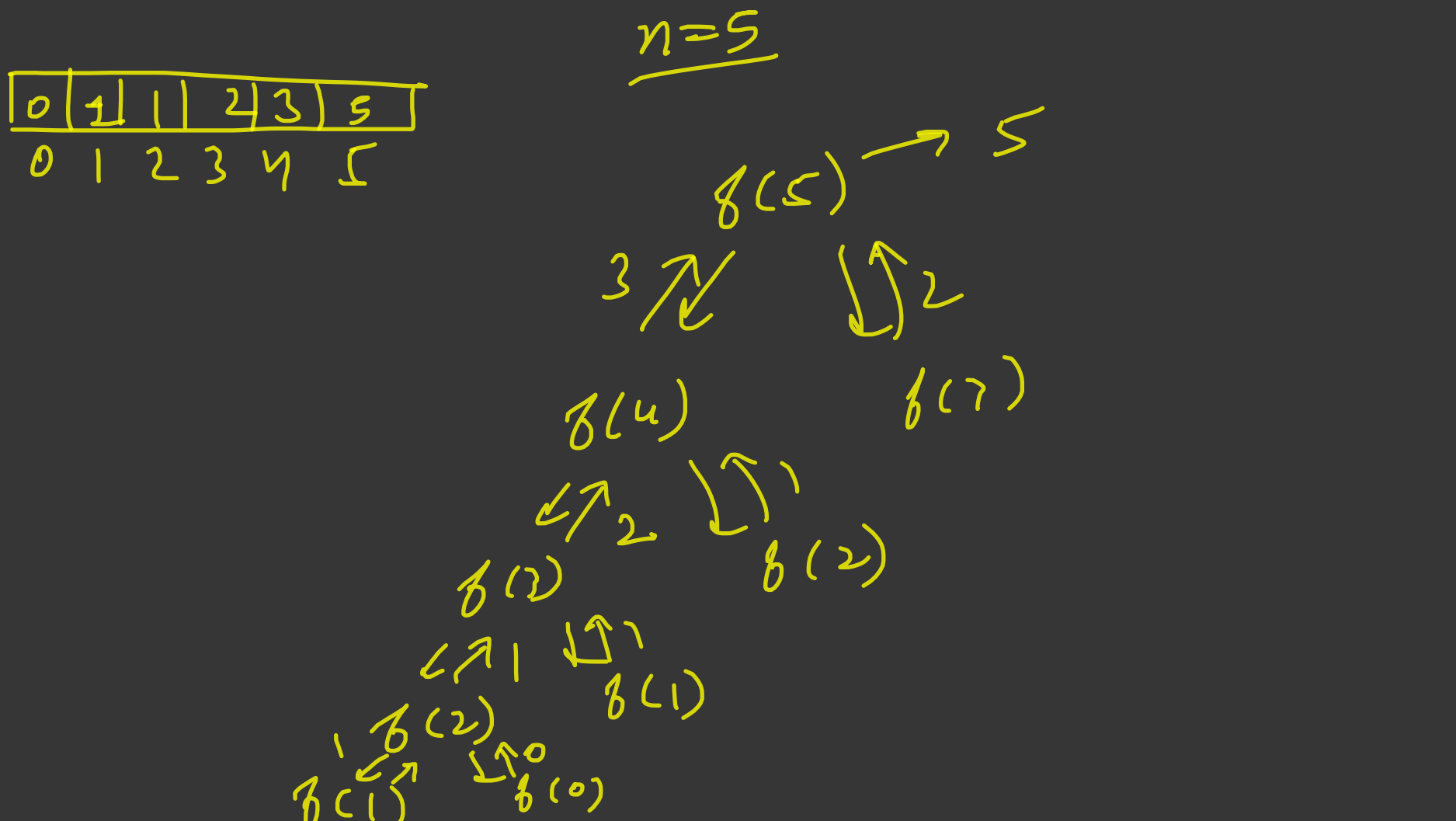

n=4
0 1 1 2 3
0 1 2 3 4
dp
f(4)
3
2
1
f(3)
f(2)
1
f(2)
f(1)
1
0
f(1)
f(0)
TC → O(n)
SC → O(n)
2^n → n
Memoization
Dynamic programming

```
int fibo(int n){
    if(n<=1) return n;
    dp[n] = fibo(n-1) + fibo(n-2);
}    return dp[n];
```

```
if(dp[n]!=-1){
    return dp[n];
}
```

recursion → dp (memoization technique)

3-4 lines of extra code


n=5
0 1 1 2 3 5
0 1 2 3 4 5
f(5)
5
3
2
f(4)
f(3)
2
1
f(2)
f(2)
1
1
f(2)
f(1)
1
0
f(1)
f(0)